

Tree-Based Methods: spam example

Li-Hsin Chien 2026-06-04

- 1 Spam Dataset
 - 1.1 Data Loading
 - 1.2 Data Splitting
- 2 Decision Tree Analysis (Classification Tree)
 - 2.1 Building the Model Using the Training Set
 - 2.2 Evaluating Model Performance Using the Testing Set
- 3 Random Forest
- 4 References

1 Spam Dataset

This dataset consists of 4,601 emails received by Hewlett-Packard (HP) over a specific period (4,601 observations). Each email is labeled as either spam (1) or non-spam (0).

Based on the email content, 57 features (covariates) have been extracted.

You can first download the data file (filename: spam.RData). The data analysis will be conducted using R software as shown below.

1.1 Data Loading

```
load("spam.RData")

colNames=c('make', 'address', 'all', '3d', 'our', 'over', 'remove', 'internet', 'order', 'm

colnames(spam)=colNames
```



1.2 Data Splitting

The dataset is partitioned into three equal parts and divided into two sets:

- Training set: Used to build and train the model, accounting for two-thirds of the data.

- Testing set: Used to evaluate the model's performance, accounting for one-third of the data.

```
set.seed(123)
# Train Test Split
train_ind <- sample(seq_len(nrow(spam)), size = 0.66*nrow(spam))
spam_train <- spam[train_ind, ]
spam_test <- spam[-train_ind, ]

spam_train_pred=spam_train[,1:57] #train: covariates (x)
spam_train_ind=spam_train[,58]    #train: outcome (y)

spam_test_pred=spam_test[,1:57]  #test: covariates (x)
spam_test_ind=spam_test[,58]     #test: outcome (y)

addmargins(table(spam_train_ind))

## spam_train_ind
## email  spam    Sum
##  1844   1192  3036
```

The training set contains a total of 3,036 samples, consisting of 1,844 non-spam emails and 1,192 spam emails.

```
addmargins(table(spam_test_ind))

## spam_test_ind
## email  spam    Sum
##   944   621  1565
```

The testing set contains a total of 1,565 samples, consisting of 944 non-spam emails and 621 spam emails.

2 Decision Tree Analysis (Classification Tree)

Load the required R packages:

```
#install.packages("rpart")
#install.packages("rpart.plot")
```

```
library(rpart)
library(rpart.plot)
library(caret)
#library(caTools)
```

2.1 Building the Model Using the Training Set

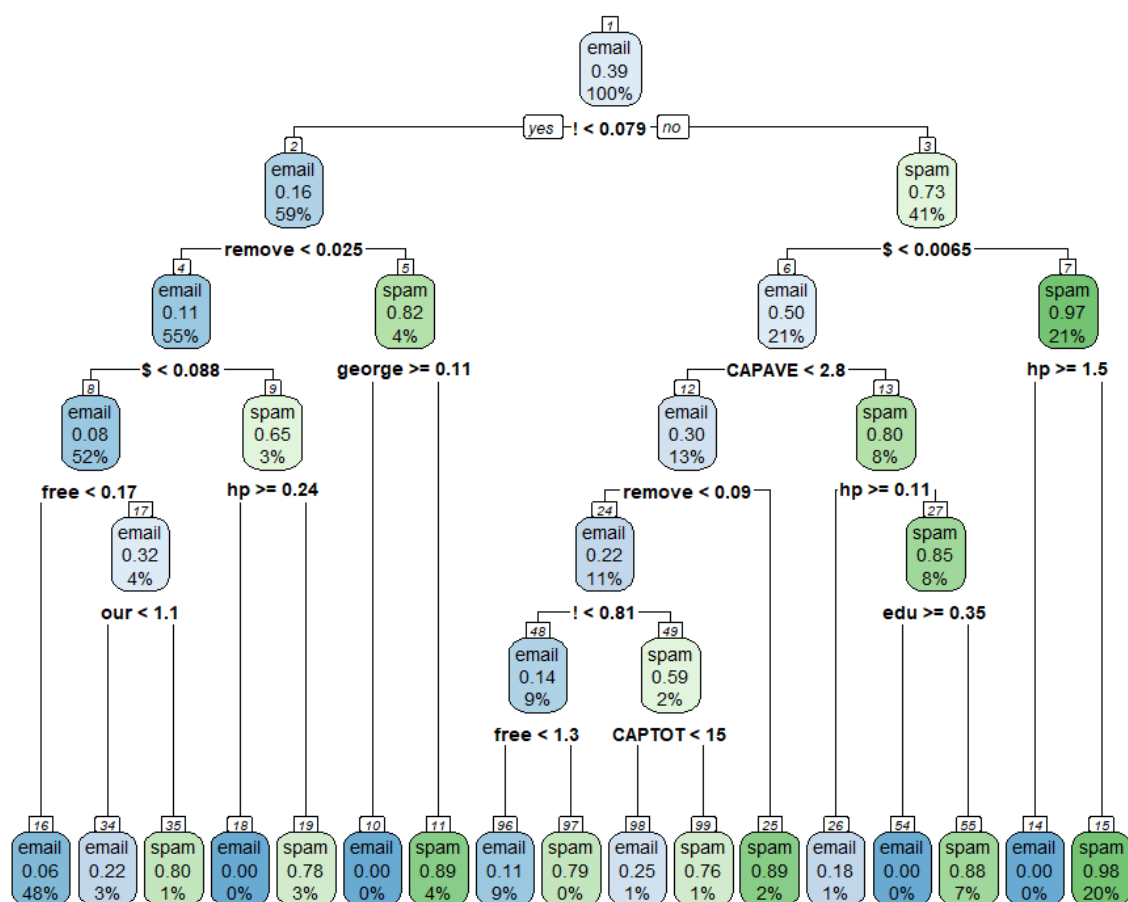
```
modelTree=rpart(as.factor(spam_train_ind)~.,data=spam_train_pred,method='class',co
#summary(modelTree)
getPred.train=predict(modelTree,newdata=spam_train_pred)[,2]
getPred.train[getPred.train>0.5]=1
getPred.train[getPred.train<=0.5]=0

spam_train_ind2<-ifelse(spam_train_ind=="spam",1,0)
confusionMatrix(as.factor(spam_train_ind2),as.factor(getPred.train))
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##           0 1752   92
##           1  138 1054
##
##           Accuracy : 0.9242
##           95% CI : (0.9143, 0.9334)
##       No Information Rate : 0.6225
##       P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa : 0.8401
##
##  McNemar's Test P-Value : 0.003005
##
##           Sensitivity : 0.9270
##           Specificity : 0.9197
##           Pos Pred Value : 0.9501
##           Neg Pred Value : 0.8842
##           Prevalence : 0.6225
##           Detection Rate : 0.5771
##       Detection Prevalence : 0.6074
##           Balanced Accuracy : 0.9234
##
##           'Positive' Class : 0
##
```

Plotting the Classification Tree:

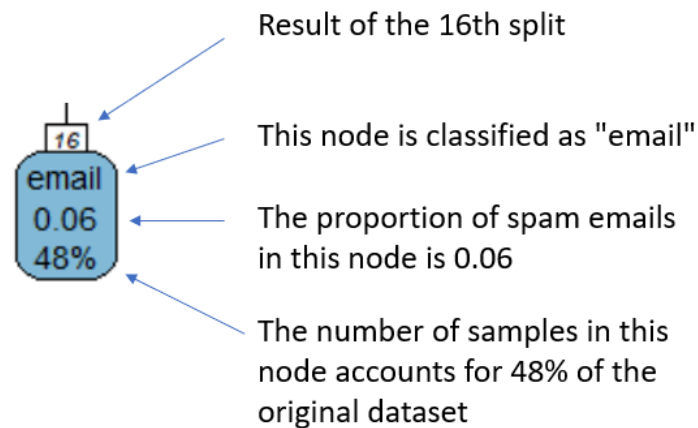
```
rpart.plot(modelTree, extra = 106, nn = TRUE)
```



The variables used in the model include:

- !: Percentage of the exclamation mark ! appearing in the email.
- remove: Percentage of the word “remove” appearing in the email.
- \$: Percentage of the dollar sign \$ appearing in the email.
- george: Percentage of the word “george” appearing in the email.
- CAPAVE: Average length of uninterrupted sequences of capital letters.
- hp: Percentage of the word “hp” appearing in the email.
- free: Percentage of the word “free” appearing in the email.
- our: Percentage of the word “our” appearing in the email.
- edu: Percentage of the word “edu” appearing in the email.
- CAPTOT: Total number of capital letters in uninterrupted sequences.

The explanation for the contents of each node is illustrated in the figure below:



2.2 Evaluating Model Performance Using the Testing Set

```
getPred=predict(modelTree,newdata=spam_test_pred)[,2]
getPred[getPred>0.5]=1
getPred[getPred<=0.5]=0
```

```
spam_test_ind2<-ifelse(spam_test_ind=="spam",1,0)
confusionMatrix(as.factor(spam_test_ind2),as.factor(getPred))
```

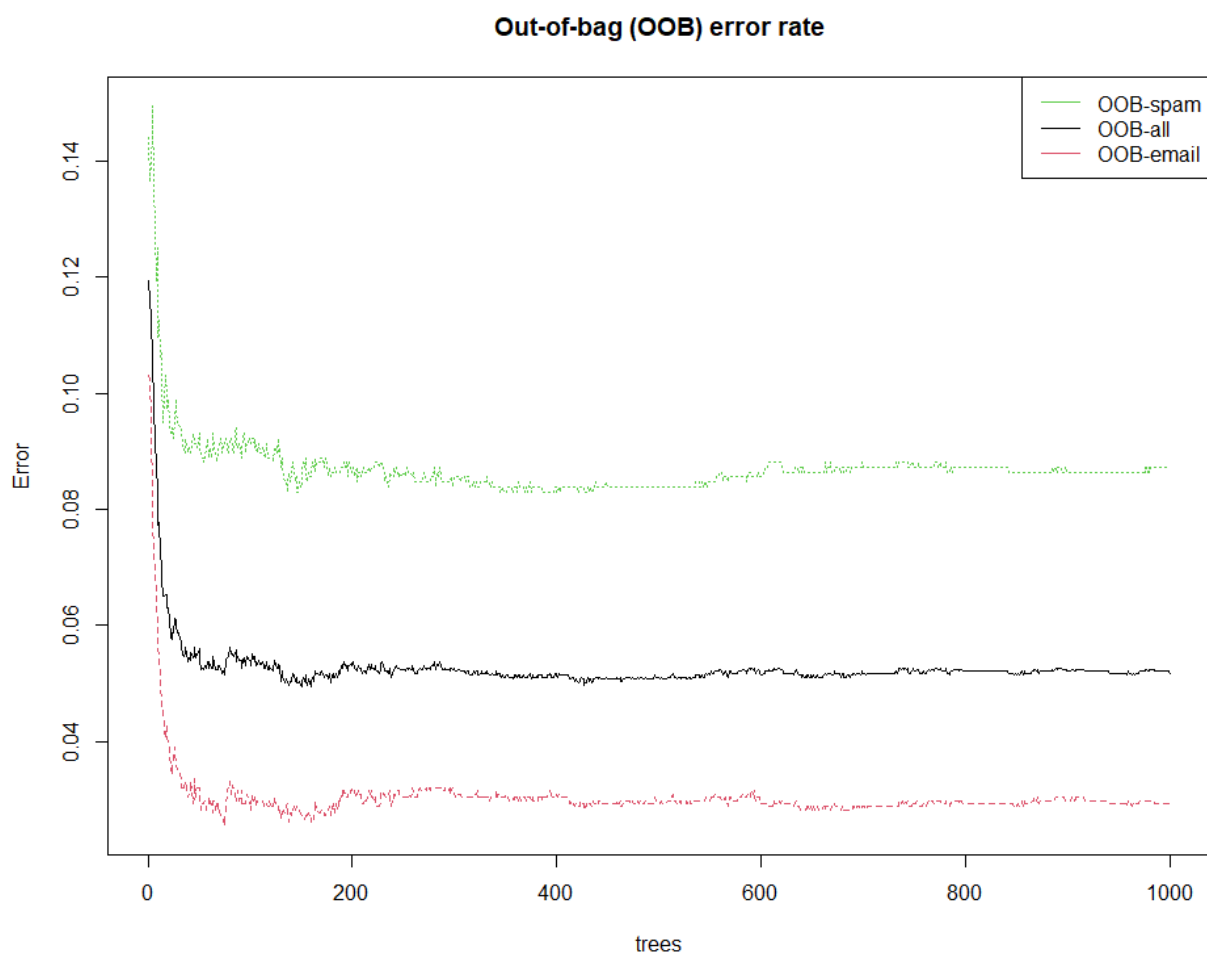
```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   0    1
##           0 885  59
##           1  72 549
##
##           Accuracy : 0.9163
##           95% CI : (0.9015, 0.9295)
##           No Information Rate : 0.6115
##           P-Value [Acc > NIR] : <2e-16
##
##           Kappa : 0.8245
##
## Mcnemar's Test P-Value : 0.2944
##
##           Sensitivity : 0.9248
##           Specificity : 0.9030
##           Pos Pred Value : 0.9375
##           Neg Pred Value : 0.8841
##           Prevalence : 0.6115
##           Detection Rate : 0.5655
##           Detection Prevalence : 0.6032
```

```
##      Balanced Accuracy : 0.9139
##
##      'Positive' Class : 0
##
```

3 Random Forest

```
library(randomForest)
```

```
modelForest=randomForest(as.factor(spam_train_ind)~.,data=data.frame(spam_train_pr
                             ntree=1000)
plot(modelForest,main="Out-of-bag (OOB) error rate")
legend("topright",c("OOB-spam", "OOB-all", "OOB-email"),col=c(3,1,2),lty=1)
```



```
pforest=predict(modelForest,newdata=data.frame(spam_test_pred))
confusionMatrix(as.factor(spam_test_ind),pforest)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction email spam
##      email   911   33
##      spam    42  579
##
##           Accuracy : 0.9521
##           95% CI : (0.9403, 0.9621)
##      No Information Rate : 0.6089
##      P-Value [Acc > NIR] : <2e-16
##
##           Kappa : 0.8996
##
##  Mcnemar's Test P-Value : 0.3556
##
##           Sensitivity : 0.9559
##           Specificity : 0.9461
##           Pos Pred Value : 0.9650
##           Neg Pred Value : 0.9324
##           Prevalence : 0.6089
##           Detection Rate : 0.5821
##      Detection Prevalence : 0.6032
##           Balanced Accuracy : 0.9510
##
##           'Positive' Class : email
##
```

4 References

- <https://search.r-project.org/CRAN/refmans/kernlab/html/spam.html>
- <https://github.com/Robby955/spam/tree/main>